App for congressional app challenge
Due 9pm

PROJECT: CIVICSEED 

Mission

Build a polished, functional, responsive web application that helps high school students turn school or community problems into evidence-based policy proposals. The app guides users through defining a problem, identifying stakeholders, collecting evidence, comparing solutions, and producing an editable policy brief.

This is a real interactive prototype, not a static mockup.

Tech stack

- Next.js with App Router
- React
- TypeScript
- CSS modules or a clean global CSS file
- Lucide icons, if available
- Browser localStorage for project persistence
- No paid APIs, backend, login, external AI API, or environment variables required

Use the existing project setup if one is already present. Do not overwrite existing files blindly. First inspect the project, preserve working code, and run the app.

Brand and visual direction

App name: CivicSeed
Tagline: From student concerns to real-world solutions.

Colors:

- Forest green: #173F35
- Sage: #B7D7A8
- Cream: #F7F3E9
- Muted blue: #8CAFC1
- White: #FFFFFF
- Body text: #29352F

Use a modern editorial design inspired by thoughtful civic-tech publications and Pinterest-style layouts. Use generous whitespace, strong typography, fine dividers, restrained color, subtle transitions, and a simple sprout logo. Use Inter or a similar sans-serif for body text and an editorial serif for major headlines.

Avoid neon gradients, excessive cards, cartoonish illustrations, emoji navigation, generic AI chatbot layouts, and excessive rounded corners.

Make the app mobile-responsive and accessible. Use semantic HTML, visible focus states, descriptive labels, and sufficient color contrast.

Required screens

1. Home

Display:

- CivicSeed wordmark and sprout icon
- Headline: "Every big change starts somewhere."
- Supporting text explaining how students can turn ideas into actionable proposals
- Primary button: "Plant your first idea"
- Continue draft section
- My Projects section
- A five-step progress indicator
- A sample project showing what a completed brief looks like

2. Define the problem

Fields:

- Project title
- Problem description
- Category: School, Accessibility, Public Health, Environment, Transportation, Youth Services, or Other
- Who is affected?
- What have you personally observed?

- What do you still need to find out?

Include an example about long school lunch lines.

3. Investigate

Allow users to:

- Add and remove evidence notes
- Enter a source title and source URL for each source
- Mark research questions complete
- Identify affected stakeholder groups
- Identify the government body, school official, or organization that might have authority to act
- List assumptions that need verification

Clearly distinguish user observations, verified evidence, and unverified assumptions. Never fabricate citations, statistics, research findings, or official authority.

4. Compare solutions

Display three possible approaches based on the problem category and the user's description. Use transparent rule-based templates rather than pretending an AI model is running.

For each approach, display:

- Proposed action
- Potential benefits
- Potential limitations
- Implementation difficulty: Low, Medium, or High
- Potential unintended consequences

Let users select an approach, edit its description, and explain why they chose it. Label suggested solutions as starting points that require research.

5. Policy brief

Generate a document dynamically using the user's actual saved answers. Include:

1. Project title
2. Executive summary
3. Problem statement
4. Affected stakeholders
5. Evidence and remaining research gaps
6. Proposed solution

7. Implementation steps
8. Potential drawbacks and safeguards
9. Measurable success indicators
10. Next steps

Include editable fields, a Copy Brief button, a Print / Save as PDF button using window.print(), and a Back to Editing button.

Do not claim the proposal has been approved, validated, or proven effective.

Core programming feature: Policy Readiness Check

Implement a real scoring function in TypeScript. Score the proposal out of 100 using five dimensions worth 20 points each:

1. Problem clarity
2. Evidence and research
3. Stakeholder consideration
4. Implementation feasibility
5. Measurable evaluation

Each dimension must have explicit scoring rules. Use field completeness and meaningful content checks, not arbitrary random numbers.

Show the total, individual dimension scores, and specific next steps. Recalculate immediately whenever relevant inputs change.

Example:

- A specific problem description improves clarity.
- Adding an evidence note and source information improves the evidence dimension.
- Identifying affected groups improves stakeholder consideration.
- Naming a decision-maker and writing implementation steps improves feasibility.
- Defining a measurable success indicator improves evaluation.

Use a score label such as "Early draft," "Developing proposal," or "Ready for further review." Do not imply that the score predicts real-world success or policy effectiveness.

Persistence and state

Create a typed project data model. Store projects in localStorage using a versioned key.

Users must be able to:

- Create a project

- Navigate between steps
- Save automatically
- Leave and reopen the browser
- Reopen an existing project
- Delete a project after confirmation
- See a helpful empty state when no projects exist

Use sensible error handling for unavailable or invalid localStorage data.

Sample project

Preload a clearly labeled demonstration project:

Title: Reducing School Lunch Wait Times

Problem: Students experience long waits during lunch, potentially reducing the time available to eat and increasing congestion.

Stakeholders:

- Students
- Cafeteria employees
- School administrators
- School board

Possible solutions:

- Improve the organization of existing serving stations
- Introduce a simple, privacy-preserving menu and queue information display
- Evaluate lunch schedules and serving capacity with staff input

These are hypothetical proposals, not established facts. Do not invent school statistics or claim that any option has been tested.

Code quality

- Use reusable components.
- Keep scoring logic and document generation in separate utility functions.
- Use TypeScript types for projects, evidence notes, solutions, and readiness scores.
- Avoid unnecessary dependencies.
- No dead buttons or fake functionality.
- Do not generate random or fabricated research.
- Keep the implementation easy for a student to understand and explain.

Completion checklist

1. Start the development server and verify that the page loads.
2. Complete the full sample workflow.
3. Verify that the readiness score changes when answers change.
4. Verify save, reload, reopen, and delete behavior.
5. Verify that the brief contains the user's actual inputs.
6. Test Copy Brief and Print / Save as PDF.
7. Test at desktop and mobile widths.
8. Fix console errors and broken interactions.
9. Explain the main files, how the score is calculated, and how the student can demonstrate the code.
10. Provide a short list of known limitations.

Prioritize a complete, reliable core workflow over extra features. If time runs short, finish the scoring, persistence, and policy brief before adding decorative details.